

Resolución de la Actividad 11: Clase 06

Grupo 4

Integrantes:

- Avila, Aldana, legajo 1193721, alavila@uade.edu.ar
- Caivano, Alexander Francisco, legajo 1209389, acaivano@uade.edu.ar
- Casareski, Juan Ignacio, legajo 1176109, jcasareski@uade.edu.ar
- Segura, Juan Ignacio, legajo 1242159, jsegura@uade.edu.ar

La resolución describe la capa de acceso del prototipo FlowOps tal como está implementada en el monorepo (backend Hono + TypeScript, repositorios en apps/backend/src/repos). Los motores del prototipo son MongoDB (definiciones e instancias), Redis (estado actual y caché de definiciones), Cassandra (eventos de auditoría) e InfluxDB (métricas). Neo4j e IRIS quedaron fuera del prototipo por decisión documentada en las actividades 7 y 9: el grafo del flujo vive como nodos y edges dentro del documento de definición en Mongo, y no hay subdominio que justifique una base de objetos.

Parte A: arquitectura de aplicación

```
Frontend (React) / Swagger UI
|
v
API FlowOps (Hono + zod-openapi)
|
+--> Validacion de request y header X-Tenant-Id (schemas zod)
|
+--> Rutas /processes
|       +--> processesRepo ---> MongoDB (cache de lectura en Redis)
|
+--> Rutas /instances
|       +--> instancesRepo ---> MongoDB
|       +--> stateRepo -----> Redis
|       +--> eventsRepo -----> Cassandra
|       +--> metricsRepo -----> InfluxDB
|
+--> Rutas /tenants
|       +--> tenantsRepo -----> MongoDB
```

El backend es demostrativo y la capa de servicio es delgada: la lógica de negocio (validar la transición contra la definición, armar la instancia, decidir el replay idempotente) vive en los handlers de las rutas. La frontera dura es el repositorio: los drivers solo se importan dentro de los módulos de repos/, y el resto de la aplicación recibe y devuelve los tipos de entidad de @flowops/types.

Capa	Responsabilidad	Qué no debería hacer
API / Controller	Validar request, params y header de tenant con zod; mapear entidades a respuestas HTTP y códigos de estado	Armar consultas de un motor o conocer claves de Redis o CQL
Service	Orquestrar repositorios para un caso de uso: validar la transición, decidir replay idempotente, registrar evento y métrica	Hablar HTTP (códigos, headers) o importar un driver
Repository / DAO	Traducir entidades a operaciones del motor y de vuelta; validar con zod en el borde donde el tipado del driver es solo una promesa	Tener lógica de negocio (qué transición vale, cuándo finaliza una instancia)
Database driver	Conexión, pooling, protocolo del motor	Ser visible fuera del repositorio

Capa	Responsabilidad	Qué no debería hacer
Cache layer	Servir lecturas repetidas (definiciones con TTL, estado actual) y reconstruirse desde la fuente de verdad	Ser fuente de verdad de ningún dato
Event / Audit layer	Registrar cada hecho de ejecución como evento inmutable e idempotente	Bloquear la operación de negocio si la métrica derivada falla

Preguntas guía:

1. ¿Dónde se valida el `tenant_id`?
En la capa API: toda ruta de datos exige el header X-Tenant-Id con un schema zod compartido (TenantHeaderSchema). Las capas de abajo lo reciben ya validado como parámetro y lo incluyen en cada filtro y clave.
2. ¿Dónde se decide qué base usar?
En ninguna parte en tiempo de ejecución: la decisión es estructural. Cada repositorio está atado a un motor, y el handler elige qué repositorio llamar según la operación. Cambiar de motor es reescribir un repositorio sin tocar handlers.
3. ¿Dónde se transforma un documento en respuesta de API?
En el handler. Los repositorios devuelven entidades tipadas (ProcessDefinition, Instance, Event), y el handler las serializa con los mismos schemas zod que generan el documento OpenAPI.
4. ¿Dónde se manejan timeouts y reintentos?
En el borde repositorio-driver: configuración de conexión del driver y política por operación dentro del repositorio (por ejemplo, consistencia ONE para el append de eventos y QUORUM para leerlos). El handler no reintenta a mano.
5. ¿Dónde se registran eventos de auditoría?
En el handler, vía eventsRepo, inmediatamente después de confirmar la escritura principal en Mongo. Nunca antes: un evento de algo que no pasó es peor que un evento tardío.

Parte B: repositories por motor

Los seis repositorios del prototipo:

Repository / DAO	Motor	Operaciones	Entrada	Salida	Riesgo principal
processes Repo	MongoDB + Redis (caché)	create, list, get	tenant_id, process_id, version opcional	Process Definition	Caché con definición vieja; se acota con TTL de 300 s y versiones inmutables
instances Repo	MongoDB	create, list, get, advance	tenant_id, instance_id, expected_step, nodo destino	Instance	Avance concurrente; lo resuelve el lock optimista por step
stateRepo	Redis	set, get	Instance / tenant_id + instance_id	Instance State	Estado desfasado si falla el SET; reconstruible desde Mongo

Repository / DAO	Motor	Operaciones	Entrada	Salida	Riesgo principal
eventsRepo	Cassandra	append, listBy Instance	Event / tenant_id + instance_id	Event[]	Timeout de escritura; el append es idempotente por (tenant, instancia, step) y se puede reintentar
metricsRepo	InfluxDB	instance Started, stepAdvanced	tenant_id, process_id, nodo, duración	void	Pérdida de puntos si Influx cae; la métrica se reconstruye desde el event log
tenantsRepo	MongoDB	list	ninguna	Tenant[]	Bajo: lectura de un catálogo chico

No hay ProcessGraphRepository: la consulta de caminos que la consigna sugiere sobre Neo4j se resuelve leyendo los edges del documento de definición. La validación “¿existe un edge del nodo actual al destino?” es un `some()` sobre un array ya cargado, no una consulta a otro motor.

Parte C: interfaces conceptuales

Interfaces de los cuatro repositorios centrales, con las firmas reales del prototipo:

```
interface ProcessDefinitionRepository {
  // Idempotente: si ya existe (tenant, process_id, version) devuelve la existente.
  create(def: ProcessDefinition): Promise<{ def: ProcessDefinition; created: boolean }>;
  list(tenantId: string): Promise<ProcessDefinition[]>;
  // Sin version devuelve la ultima. Con version pasa por el cache Redis.
  get(tenantId: string, processId: string, version?: number): Promise<ProcessDefinition | null>;
}

interface InstanceRepository {
  create(inst: Instance): Promise<{ inst: Instance; created: boolean }>;
  list(tenantId: string, limit?: number): Promise<Instance[]>;
  get(tenantId: string, instanceId: string): Promise<Instance | null>;
  // Lock optimista: solo avanza si la instancia sigue en expectedStep y corriendo.
  advance(
    tenantId: string,
    instanceId: string,
    expectedStep: number,
    to: string,
    status: InstanceStatus,
  ): Promise<Instance | null>;
}

interface EventLogRepository {
  // Clave (tenant, instancia, step): reinsertar el mismo evento es un upsert.
  append(event: Event): Promise<void>;
  listByInstance(tenantId: string, instanceId: string): Promise<Event[]>;
}

interface StateCacheRepository {
  set(inst: Instance): Promise<void>;
  get(tenantId: string, instanceId: string): Promise<InstanceState | null>;
}
```

Interfaz	Métodos	Motor detrás	Qué abstracción aporta
ProcessDefinition Repository	create, list, get	MongoDB y Redis	Oculto dos motores detrás de un método: el caller no sabe si la definición vino del caché o de Mongo
InstanceRepository	create, list, get, advance	MongoDB	Encierra el lock optimista en una operación atómica; el handler no arma filtros de Mongo
EventLogRepository	append, listBy Instance	Cassandra	Oculto CQL, niveles de consistencia por operación y la validación zod de cada row
StateCacheRepository	set, get	Redis	Oculto el armado de claves y el parse + validación del JSON guardado

En los cuatro casos la dependencia que queda escondida es el driver, el dialecto de consulta (filtros de Mongo, CQL, claves de Redis, line protocol) y la política de validación: donde el tipado del driver es solo un cast (Redis, Cassandra con execute, Influx), el repositorio valida con el schema zod de la entidad antes de devolver.

Parte D: flujo de iniciar una instancia

Flujo real de POST /instances en el prototipo:

Paso	Componente	Acción	Base involucrada	Error posible
1	API	Valida body y header X-Tenant-Id con zod	Ninguna	Request inválido: 400
2	processes Repo	Busca la definición en el caché	Redis	Miss o Redis caído: sigue al paso 3
3	processes Repo	Lee la definición de Mongo y repuebla el caché	MongoDB	No existe: 404
4	Handler	Ubica el nodo start y arma la instancia en step 0	Ninguna	Definición sin nodo start: 400
5	instances Repo	Inserta la instancia; índice único por (tenant, instance_id)	MongoDB	Clave duplicada: replay idempotente, responde 200 con la existente
6	eventsRepo	Registra instance_started con consistencia ONE	Cassandra	Timeout: reintento seguro (append idempotente)
7	stateRepo	Guarda el estado actual con SET	Redis	Falla: el estado se reconstruye en la próxima lectura
8	metricsRepo + API	Escribe el punto instance_started y responde 201 con instance_id, current_node y step	InfluxDB	Influx caído: la métrica se pierde y se reconstruye luego del event log

Los pasos 6, 7 y 8 corren solo cuando la instancia se creó de verdad: el replay del paso 5 devuelve la instancia existente sin duplicar evento, estado ni métrica.

Parte E: manejo de errores

Escenario	Qué puede fallar	Estrategia	Qué se informa al usuario
MongoDB no responde al buscar definición	Conexión o timeout del driver	Cortar la operación: Mongo es la fuente de verdad y sin definición no se puede validar nada. Reintento corto del driver y error	503, servicio no disponible, reintentar
Redis no responde al consultar estado cacheado	Conexión caída, no solo un miss	Degradar a Mongo: el mismo camino del cache miss ya implementado (leer la instancia y repoblar Redis)	Nada: respuesta correcta, algo más lenta
Cassandra no puede registrar evento	Timeout o nodo caído	Reintentar con backoff: el append es idempotente por (tenant, instancia, step), así que repetirlo no duplica. Si se agota, registrar el hueco y re-emitar después desde el estado en Mongo	Éxito de la operación de negocio, que ya se confirmó en Mongo
Neo4j no puede validar camino del flujo	No aplica al prototipo	La validación de transición lee los edges del documento de definición en Mongo; la falla posible es la del primer escenario	Igual que el primer escenario
El usuario intenta completar dos veces la misma tarea	Doble click o reintento del cliente sobre el avance	Idempotencia por expected_step: si el avance ya se aplicó se responde lo mismo sin escribir; si el step no coincide con el actual ni el anterior, se rechaza	200 con el mismo resultado, o 409 si el estado real avanzó por otro lado
El tenant no existe o está deshabilitado	Header con tenant inválido	Cortar en la capa API antes de tocar repositorios. En el prototipo el front solo ofrece los tenants existentes y toda clave y filtro incluye el tenant, así que un tenant inexistente solo ve vacío	404, tenant desconocido

Preguntas guía:

1. ¿Qué fallas permiten continuar?
Las de Redis (caché y estado: ambos se reconstruyen desde Mongo) y las de InfluxDB (la métrica es un dato derivado del event log).
2. ¿Qué fallas obligan a cortar la operación?
Las de MongoDB. Es la fuente de verdad de definiciones e instancias: sin ella no se puede validar una transición ni confirmar una escritura.
3. ¿Qué fallas requieren reintento?
Los timeouts transitorios de Cassandra e InfluxDB. Como todas las escrituras son idempotentes (clave derivada del step, punto con timestamp fijo), reintentar es seguro.
4. ¿Qué fallas requieren registrar auditoría?
Toda operación de negocio confirmada debe terminar con su evento en Cassandra; si el append falla, el hueco queda pendiente de re-emisión. Los rechazos por conflicto (409) no escriben evento porque no cambió nada.
5. ¿Qué datos se pueden reconstruir luego?
El estado en Redis y el caché de definiciones (desde Mongo) y las métricas (desde el event log). No se reconstruyen los documentos de Mongo ni los eventos de Cassandra: son las dos fuentes de verdad.
6. ¿Dónde se usaría circuit breaker conceptual?
Delante de InfluxDB: tras varias fallas seguidas se deja de intentar y se descartan o encolan métricas un rato, sin frenar el flujo. También delante del caché de Redis: con Redis caído conviene ir directo a Mongo un tiempo en vez de pagar un timeout por request.

Parte F: sincronización entre bases

La regla del prototipo: primero se confirma la escritura en la fuente de verdad (Mongo) y recién después se escriben las derivadas, en secuencia. Las derivadas toleran atraso porque toda lectura crítica tiene fallback a la fuente de verdad, y las re-escrituras son idempotentes.

Evento de dominio	Base principal	Bases derivadas	Sincrónico o asincrónico	Riesgo de inconsistencia
Proceso publicado	MongoDB	Redis (caché de definición, TTL 300 s, se puebla en la primera lectura)	Sincrónico solo Mongo	Casi nulo: las definiciones son inmutables por versión, una versión nueva usa una clave de caché nueva
Instancia iniciada	MongoDB	Cassandra (evento), Redis (estado), InfluxDB (métrica)	Sincrónico en el prototipo; la métrica admite ser asincrónica	Falla entre Mongo y Cassandra deja el evento faltante; se re-emite con la misma clave sin duplicar
Paso avanzado (la tarea completada del modelo, el avance es manual)	MongoDB (lock optimista por step)	Cassandra, Redis, InfluxDB	Sincrónico tras confirmar el avance	Estado viejo en Redis si falla el SET; la próxima lectura lo reconstruye desde Mongo

Evento de dominio	Base principal	Bases derivadas	Sincrónico o asincrónico	Riesgo de inconsistencia
Error de ejecución	No se persiste como estado	Ninguna	No aplica	Las transiciones inválidas se rechazan antes de escribir (400 o 409), así que no dejan escrituras parciales
Proceso finalizado	MongoDB (status finished)	Cassandra (evento instance_finished), Redis, InfluxDB	Sincrónico	Igual que el avance; una instancia finalizada ya no acepta escrituras, lo que cierra la ventana de carrera

Parte G: API unificada

Endpoints reales del prototipo. A diferencia de la estructura sugerida `/api/v1/{tenant_id}/...`, el tenant no viaja en la URL: va en el header `X-Tenant-Id`, que toda ruta de datos exige (decisión multi-tenant de la Actividad 5). Las URLs quedan más cortas y el tenant se valida una sola vez.

Endpoint	Método	Qué hace	Servicios internos	Bases involucradas
<code>/processes</code>	POST	Crea una definición de proceso (idempotente por tenant + process_id + version)	processesRepo	MongoDB
<code>/processes/{process_id}</code>	GET	Consulta una definición (última o por versión)	processesRepo	Redis, MongoDB
<code>/instances</code>	POST	Inicia una instancia	processesRepo, instancesRepo, eventsRepo, stateRepo, metricsRepo	Las cuatro
<code>/instances/{instance_id}/state</code>	GET	Estado rápido de la instancia, con fallback	stateRepo, instancesRepo	Redis, MongoDB
<code>/instances/{instance_id}/advance</code>	POST	Avanza la instancia al siguiente nodo con lock optimista	processesRepo, instancesRepo, eventsRepo, stateRepo, metricsRepo	Las cuatro
<code>/instances/{instance_id}/events</code>	GET	Historial de auditoría de la instancia (lectura QUORUM)	eventsRepo, instancesRepo	Cassandra, MongoDB

Parte H: evidencia para el TPO

Evidencia	Qué demuestra	Cómo se mostraría
Diagrama de arquitectura	Que cada motor tiene un único punto de entrada y los drivers no salen de los repositorios	El diagrama de la parte A junto al árbol de apps/backend/src
Tabla repository-motor	Que las responsabilidades no se solapan: un repositorio por motor, operaciones con tipos de entidad	La tabla de la parte B y el código de repos/
Endpoint de inicio de instancia	El flujo políglota completo: Mongo, Cassandra, Redis e Influx en una operación	Swagger UI: POST /instances responde 201; repetirlo con el mismo instance_id responde 200 sin duplicar
Consulta de eventos	Que la auditoría es inmutable y se lee por instancia	GET /instances/{id}/events tras varios avances, y la misma partición vista en cqlsh (pnpm infra view:cassandra)
Falla simulada de Redis	Que Redis no es fuente de verdad	Bajar el contenedor de Redis: la consulta del documento completo sigue saliendo de Mongo; al volver Redis vacío, la primera consulta de estado lo reconstruye y lo repuebla
Log de auditoría	La clave (tenant, instancia, step) y el upsert idempotente	SELECT en cqlsh antes y después de reintentar un avance: misma cantidad de filas
Captura de base NoSQL	Que cada dato vive en el motor decidido	pnpm infra view:mongodb (Mongo Express), view:redis (Redis Commander), view:influxdb (UI de Influx) sobre los datos del seed